

Hitters Tree and Ensemble Regression

Project Overview

This project analyzes the **Hitters dataset**, which contains Major League Baseball player statistics and salary information. The goal is to study the relationship between player characteristics and salary using tree-based regression and ensemble learning methods.

The analysis includes:

- Regression Tree
- Tree Pruning
- Bagging
- Random Forest
- Boosting
- Leave-One-Out Cross-Validation (LOOCV) for estimating test MSE

Dataset

The Hitters dataset contains information about Major League Baseball players, including their batting performance, career statistics, league information, and salary.

The response variable is **Salary**. Because salary is right-skewed, **log(Salary)** is used as the response variable in the regression models.

After removing observations with missing salary values, the dataset contains **263 players** and **19 predictor variables**.

The predictors include:

- Current-season performance, such as AtBat, Hits, HmRun, Runs, RBI, and Walks
- Career performance, such as CAtBat, CHits, CHmRun, CRuns, CRBI, and CWalks
- Player and team information, such as Years, League, Division, PutOuts, Assists, Errors, and NewLeague

Objective

The objective is to compare several tree-based regression methods and determine which approach provides the lowest estimated test mean squared error (MSE). Variable importance will also be examined to identify which player characteristics are most useful for explaining salary.

```
In [1]: # Import libraries for data manipulation and numerical operations
import pandas as pd
import numpy as np
```

```
C:\Users\shrup\anaconda3\lib\site-packages\pandas\core\computation\expressions.py:21:
UserWarning: Pandas requires version '2.8.4' or newer of 'numexpr' (version '2.8.3' c
urrently installed).
    from pandas.core.computation.check import NUMEXPR_INSTALLED
C:\Users\shrup\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:61: UserWarni
ng: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' current
ly installed).
    from pandas.core import (
```

```
In [2]: from ISLP import load_data
```

```
In [3]: # Load the Hitters dataset
Hitters = load_data("Hitters")

# Display the first five rows
Hitters.head()
```

```
Out[3]:
```

	AtBat	Hits	HmRun	Runs	RBI	Walks	Years	CAAtBat	CHits	CHmRun	CRuns	CRBI	CWalks	Le
0	293	66	1	30	29	14	1	293	66	1	30	29	14	
1	315	81	7	24	38	39	14	3449	835	69	321	414	375	
2	479	130	18	66	72	76	3	1624	457	63	224	266	263	
3	496	141	20	65	78	37	11	5628	1575	225	828	838	354	
4	321	87	10	39	42	30	2	396	101	12	48	46	33	

```
In [4]: # Check the dimensions of the dataset
Hitters.shape
```

```
Out[4]: (322, 20)
```

```
In [5]: # Check the number of missing Salary values
Hitters["Salary"].isna().sum()
```

```
Out[5]: 59
```

```
In [6]: # Remove observations with missing Salary
Hitters = Hitters.dropna(subset=["Salary"])

# Check the new dimensions
Hitters.shape
```

```
Out[6]: (263, 20)
```

```
In [7]: # Create log-transformed Salary
Hitters["log_Salary"] = np.log(Hitters["Salary"])
```

```
In [8]: Hitters[["Salary", "log_Salary"]].head()
```

Out[8]:

	Salary	log_Salary
1	475.0	6.163315
2	480.0	6.173786
3	500.0	6.214608
4	91.5	4.516339
5	750.0	6.620073

```
In [9]: # Response variable
y = Hitters["log_Salary"]

# Predictor variables
X = Hitters.drop(columns=["Salary", "log_Salary"])
print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (263, 19)
y shape: (263,)
```

```
In [10]: # Check data types of all predictors
X.dtypes
```

```
Out[10]: AtBat          int64
Hits            int64
HmRun           int64
Runs            int64
RBI             int64
Walks           int64
Years           int64
CAtBat          int64
CHits           int64
CHmRun          int64
CRuns           int64
CRBI            int64
CWalks          int64
League          category
Division        category
PutOuts         int64
Assists         int64
Errors          int64
NewLeague       category
dtype: object
```

```
In [11]: # Convert categorical predictors into dummy variables
X = pd.get_dummies(X, drop_first=True, dtype=int)

# Check the dimensions
print("X shape:", X.shape)

# Display the first 5 rows
X.head()
```

X shape: (263, 19)

```
Out[11]:
```

	AtBat	Hits	HmRun	Runs	RBI	Walks	Years	CAtBat	CHits	CHmRun	CRuns	CRBI	CWalks	PI
1	315	81	7	24	38	39	14	3449	835	69	321	414	375	
2	479	130	18	66	72	76	3	1624	457	63	224	266	263	
3	496	141	20	65	78	37	11	5628	1575	225	828	838	354	
4	321	87	10	39	42	30	2	396	101	12	48	46	33	
5	594	169	4	74	51	35	11	4408	1133	19	501	336	194	

Regression Tree

◀ A regression tree is fitted using all 19 predictors with `log(Salary)` as the response variable. ▶

```
In [12]: from sklearn.tree import DecisionTreeRegressor
```

```
In [13]: # Create the regression tree
tree_model = DecisionTreeRegressor(random_state=1)

# Fit the tree using all predictors
tree_model.fit(X, y)
```

```
Out[13]:
```

DecisionTreeRegressor ⓘ ?

DecisionTreeRegressor(random_state=1)

```
In [14]: import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

```
In [ ]: # Plot the regression tree
plt.figure(figsize=(20, 10))

plot_tree(
    tree_model,
    feature_names=X.columns,
    filled=True,
    rounded=True,
    fontsize=7
)

plt.title("Regression Tree for log(Salary)")

# Save figure
plt.savefig(
    "Figures/unpruned_regression_tree.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

LOOCV Test MSE

```
In [ ]: from sklearn.model_selection import LeaveOneOut, cross_val_score
```

```
In [ ]: # Create Leave-One-Out Cross-Validation  
loo = LeaveOneOut()
```

```
In [ ]: # Perform LOOCV on the unpruned regression tree  
cv_scores = cross_val_score(  
    tree_model,  
    X,  
    y,  
    cv=loo,  
    scoring="neg_mean_squared_error"  
)  
  
# Convert negative MSE scores to positive and calculate the average  
loocv_mse = -cv_scores.mean()  
  
print("LOOCV Test MSE:", loocv_mse)
```

Results

The unpruned regression tree produced an estimated LOOCV test MSE of **0.3318** on the log(Salary) scale.

The first split of the tree was based on **CAtBat (Career At Bats)** at 1452. Players with CAtBat $\leq$ 1452 had a lower average log(Salary), while players with CAtBat $>$ 1452 had a higher average log(Salary). Subsequent splits included variables such as CHits, Hits, PutOuts, and CRBI.

The unpruned tree contains many terminal nodes, including nodes with very few observations, indicating that the tree is highly complex and may be overfitting the training data. Pruning will therefore be evaluated using LOOCV to determine whether a smaller tree can improve estimated test performance.

Pruning the Regression Tree

Cost-complexity pruning is used to determine whether a smaller tree can improve predictive performance. Leave-One-Out Cross-Validation (LOOCV) is used to select the optimal pruning parameter.

```
In [ ]: # Get the cost-complexity pruning path  
path = tree_model.cost_complexity_pruning_path(X, y)  
  
ccp_alphas = path.ccp_alphas  
  
# Check how many candidate alpha values there are  
len(ccp_alphas)
```

```
In [ ]: # Store LOOCV MSE for each alpha  
loocv_mse_pruning = []  
  
for alpha in ccp_alphas:
```

```

# Create a tree with this pruning parameter
tree = DecisionTreeRegressor(
    random_state=1,
    ccp_alpha=alpha
)

# Calculate LOOCV MSE
scores = cross_val_score(
    tree,
    X,
    y,
    cv=loo,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

# Store positive MSE
loocv_mse_pruning.append(-scores.mean())

```

```

In [ ]: # Find the alpha with the minimum LOOCV MSE
best_index = np.argmin(loocv_mse_pruning)

best_alpha = ccp_alphas[best_index]
best_mse = loocv_mse_pruning[best_index]

print("Best ccp_alpha:", best_alpha)
print("Minimum LOOCV MSE:", best_mse)

```

```

In [ ]: # Fit the optimally pruned regression tree
pruned_tree = DecisionTreeRegressor(
    random_state=1,
    ccp_alpha=best_alpha
)

pruned_tree.fit(X, y)

```

```

In [ ]: # Number of terminal nodes
print("Unpruned tree terminal nodes:", tree_model.get_n_leaves())
print("Pruned tree terminal nodes:", pruned_tree.get_n_leaves())

# Tree depth
print("Unpruned tree depth:", tree_model.get_depth())
print("Pruned tree depth:", pruned_tree.get_depth())

```

```

In [ ]: # Plot the pruned regression tree
plt.figure(figsize=(16, 8))

plot_tree(
    pruned_tree,
    feature_names=X.columns,
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title("Pruned Regression Tree for log(Salary)")
# Save the figure

```

```
plt.savefig(
    "Figures/pruned_regression_tree.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Terminal Node Regions

The optimally pruned tree contains six terminal nodes, corresponding to the following regions:

- **R1:** $\text{CAtBat} \leq 1452$, $\text{CHits} \leq 182$, and $\text{CRuns} \leq 11$
- **R2:** $\text{CAtBat} \leq 1452$, $\text{CHits} \leq 182$, and $\text{CRuns} > 11$
- **R3:** $\text{CAtBat} \leq 1452$ and $\text{CHits} > 182$
- **R4:** $\text{CAtBat} > 1452$ and $\text{Hits} \leq 117.5$
- **R5:** $\text{CAtBat} > 1452$, $\text{Hits} > 117.5$, and $\text{CRBI} \leq 273$
- **R6:** $\text{CAtBat} > 1452$, $\text{Hits} > 117.5$, and $\text{CRBI} > 273$

Pruning Results

LOOCV selected an optimal cost-complexity parameter of **ccp_alpha = 0.01167**. The resulting pruned tree contains **6 terminal nodes** with a depth of **3**, compared with 245 terminal nodes and a depth of 15 for the unpruned tree.

The estimated LOOCV test MSE decreased from **0.3318** for the unpruned tree to **0.2278** for the pruned tree. Therefore, pruning was helpful because it produced a substantially simpler tree while also improving estimated test performance.

The pruned tree primarily uses career and batting performance variables, including CAtBat, CHits, CRuns, Hits, and CRBI, suggesting that these variables are important for explaining differences in $\log(\text{Salary})$.

```
In [ ]: # Plot LOOCV MSE against pruning parameter
plt.figure(figsize=(8, 5))

plt.plot(ccp_alphas, loocv_mse_pruning, marker="o", markersize=3)

# Mark the optimal alpha
plt.axvline(
    best_alpha,
    linestyle="--",
    label=f"Best alpha = {best_alpha:.4f}"
)

plt.xlabel("Cost-Complexity Parameter (ccp_alpha)")
plt.ylabel("LOOCV Test MSE")
plt.title("LOOCV MSE vs. Tree Pruning Parameter")
plt.legend()

# Save figure
plt.savefig(
```

```

    "Figures/pruning_loocv_mse.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```

Interpretation

The plot shows how the LOOCV test MSE changes as the cost-complexity parameter (`ccp_alpha`) increases. The minimum LOOCV MSE occurs at approximately **`ccp_alpha = 0.0117`**, with an estimated test MSE of **0.2278**.

Small values of `ccp_alpha` produce more complex trees, while larger values result in more pruning. After the optimal value, the LOOCV MSE increases, indicating that excessive pruning removes useful information and leads to underfitting.

Therefore, **`ccp_alpha = 0.0117`** was selected for the final pruned regression tree.

Bagging

A bagging regression model with 1,000 regression trees is fitted using all 19 predictors. LOOCV is used to estimate the test MSE.

```
In [ ]: from sklearn.ensemble import RandomForestRegressor
```

```
In [ ]: # Bagging model with B = 1000 trees
bagging_model = RandomForestRegressor(
    n_estimators=1000,
    max_features=1.0,
    random_state=1,
    n_jobs=-1
)
```

```
In [ ]: # Perform LOOCV for the bagging model
bagging_scores = cross_val_score(
    bagging_model,
    X,
    y,
    cv=loo,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

# Convert to positive MSE
bagging_mse = -bagging_scores.mean()

print("Bagging LOOCV Test MSE:", bagging_mse)
```

```
In [ ]: # Fit bagging model to the full dataset
bagging_model.fit(X, y)
```



```
In [ ]: # Get feature importance
bagging_importance = pd.DataFrame({
    "Predictor": X.columns,
    "Importance": bagging_model.feature_importances_
})

# Sort from most important to Least important
bagging_importance = bagging_importance.sort_values(
    by="Importance",
    ascending=False
)

bagging_importance
```

```
In [ ]: # Select the 10 most important predictors
top10_bagging = bagging_importance.head(10)

# Plot feature importance
plt.figure(figsize=(8, 5))

plt.barh(
    top10_bagging["Predictor"],
    top10_bagging["Importance"]
)

# Put the most important predictor at the top
plt.gca().invert_yaxis()

plt.xlabel("Feature Importance")
plt.ylabel("Predictor")
plt.title("Top 10 Predictor Importances - Bagging")

# Save figure
plt.savefig(
    "Figures/bagging_feature_importance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Bagging Results

The bagging model with 1,000 trees produced an estimated LOOCV test MSE of **0.1888**, which is lower than the MSE of both the unpruned tree (0.3318) and the pruned tree (0.2278).

The most important predictors in the bagging model were **CAtBat (Career At Bats)**, **CRuns (Career Runs)**, and **CHits (Career Hits)**. CAtBat had the highest feature importance, indicating that career batting experience was particularly influential in predicting log(Salary).

Overall, bagging improved predictive performance compared with the individual regression trees.

Random Forest

A random forest regression model with 1,000 trees is fitted using approximately one-third of the predictors at each split. Leave-One-Out Cross-Validation (LOOCV) is used to estimate the test MSE.

```
In [ ]: # Random Forest with B = 1000 and m ≈ p/3
random_forest_model = RandomForestRegressor(
    n_estimators=1000,
    max_features=6,
    random_state=1,
    n_jobs=-1
)
```

```
In [ ]: # Perform LOOCV for the random forest model
rf_scores = cross_val_score(
    random_forest_model,
    X,
    y,
    cv=loo,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

# Convert to positive MSE
rf_mse = -rf_scores.mean()

print("Random Forest LOOCV Test MSE:", rf_mse)
```

```
In [ ]: # Fit Random Forest using the full dataset
random_forest_model.fit(X, y)
```

```
In [ ]: # Create feature importance table
rf_importance = pd.DataFrame({
    "Predictor": X.columns,
    "Importance": random_forest_model.feature_importances_
})

# Sort from most important to least important
rf_importance = rf_importance.sort_values(
    by="Importance",
    ascending=False
)

rf_importance
```

```
In [ ]: # Select the 10 most important predictors
top10_rf = rf_importance.head(10)
```

```
In [ ]: # Plot Random Forest feature importance
plt.figure(figsize=(8, 5))

plt.barh(
    top10_rf["Predictor"],
    top10_rf["Importance"]
)

# Put the most important predictor at the top
```

```
plt.gca().invert_yaxis()

plt.xlabel("Feature Importance")
plt.ylabel("Predictor")
plt.title("Top 10 Predictor Importances - Random Forest")

# Save figure
plt.savefig(
    "Figures/random_forest_feature_importance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Random Forest Results

The random forest model with 1,000 trees and approximately one-third of the predictors considered at each split produced a LOOCV test MSE of **0.1800**.

This is slightly lower than the bagging LOOCV test MSE of **0.1888**, indicating that random forest provided better estimated test performance.

The most important predictors were **CAtBat**, **CHits**, **CRuns**, **CRBI**, and **CWalks**. Most of the highest-ranked predictors are career statistics, suggesting that long-term batting performance is particularly important for predicting log(Salary).

Boosting

A gradient boosting regression model is fitted using 1,000 trees, a maximum tree depth of 1, and a learning rate of 0.01. LOOCV is used to estimate the test MSE.

```
In [ ]: from sklearn.ensemble import GradientBoostingRegressor
```

```
In [ ]: # Boosting model: B = 1000, d = 1, Lambda = 0.01
boosting_model = GradientBoostingRegressor(
    n_estimators=1000,
    max_depth=1,
    learning_rate=0.01,
    random_state=1
)
```

```
In [ ]: # Perform LOOCV for the boosting model
boosting_scores = cross_val_score(
    boosting_model,
    X,
    y,
    cv=loo,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

# Convert to positive MSE
```

```
boosting_mse = -boosting_scores.mean()

print("Boosting LOOCV Test MSE:", boosting_mse)
```

```
In [ ]: # Fit boosting model using the full dataset
boosting_model.fit(X, y)
```

```
In [ ]: # Create feature importance table
boosting_importance = pd.DataFrame({
    "Predictor": X.columns,
    "Importance": boosting_model.feature_importances_
})

# Sort from most important to Least important
boosting_importance = boosting_importance.sort_values(
    by="Importance",
    ascending=False
)

boosting_importance
```

```
In [ ]: # Select the 10 most important predictors
top10_boosting = boosting_importance.head(10)
```

```
In [ ]: # Plot Boosting feature importance
plt.figure(figsize=(8, 5))

plt.barh(
    top10_boosting["Predictor"],
    top10_boosting["Importance"]
)

# Put the most important predictor at the top
plt.gca().invert_yaxis()

plt.xlabel("Feature Importance")
plt.ylabel("Predictor")
plt.title("Top 10 Predictor Importances - Boosting")

# Save figure
plt.savefig(
    "Figures/boosting_feature_importance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Boosting Results

The boosting model with 1,000 trees, a maximum depth of 1, and a learning rate of 0.01 produced a LOOCV test MSE of **0.1928**.

The most important predictors were **CAtBat**, **CHits**, and **CRBI**, with CAtBat having the highest feature importance. Overall, career batting statistics appear to play an important role in predicting log(Salary).

The boosting model performed better than the individual regression trees but did not outperform bagging or random forest.

Model Comparison

```
In [ ]: # Create a comparison table of LOOCV test MSE
model_comparison = pd.DataFrame({
    "Method": [
        "Unpruned Regression Tree",
        "Pruned Regression Tree",
        "Bagging",
        "Random Forest",
        "Boosting"
    ],
    "LOOCV Test MSE": [
        loocv_mse,
        best_mse,
        bagging_mse,
        rf_mse,
        boosting_mse
    ]
})

# Sort models from lowest to highest MSE
model_comparison = model_comparison.sort_values(
    by="LOOCV Test MSE"
)

model_comparison
```

Model Comparison and Recommendation

Among the tree-based and ensemble methods considered, **Random Forest** achieved the lowest LOOCV test MSE of **0.1800**, followed by Bagging (0.1888), Boosting (0.1928), the Pruned Regression Tree (0.2278), and the Unpruned Regression Tree (0.3318).

Therefore, **Random Forest is recommended** among the methods evaluated in this project because it produced the lowest estimated test error.

Pruning substantially improved the performance of the individual regression tree, reducing the LOOCV test MSE from 0.3318 to 0.2278. The ensemble methods improved performance further, with Random Forest providing the best overall result.

In the previous project, **Lasso regression** was recommended among the regression and variable-selection methods considered. In the current project, Random Forest is preferred among the tree-based and ensemble methods. Thus, the recommended method changes from a regularized linear regression approach to an ensemble tree-based approach.

Conclusion

This project compared regression trees and ensemble learning methods for predicting **log(Salary)** using the Hitters dataset. Model performance was evaluated using Leave-One-Out Cross-Validation (LOOCV).

Pruning improved the regression tree by reducing the LOOCV test MSE from **0.3318** to **0.2278**, while reducing the number of terminal nodes from 245 to 6.

The ensemble methods provided better predictive performance than the individual regression trees:

- **Bagging:** MSE = 0.1888
- **Random Forest:** MSE = 0.1800
- **Boosting:** MSE = 0.1928

Among all methods, **Random Forest achieved the lowest LOOCV test MSE (0.1800)** and was therefore selected as the best-performing model.

Career batting statistics, particularly **CAtBat**, **CHits**, **CRuns**, and **CRBI**, were consistently identified as important predictors of player salary.

Overall, the results demonstrate that ensemble tree methods can provide better predictive performance than a single regression tree.